

The Classic Password Validator

Company: Accenture

Round: Online Assessment (OA)

Year: 2025

Difficulty: Easy

Topic: Strings, Conditional Logic, Validation

Source: [Accenture Coding OA question](#)

Problem Description

You are given a function `CheckPassword(str, n)` that accepts a string `str` of length `n`. Implement the function to return 1 if the string is a valid password, otherwise return 0. A password is valid if it satisfies **all** of the following conditions:

- It must be at least 4 characters long.
- It must contain at least one numeric digit.
- It must contain at least one capital letter.
- It must not have any spaces or slashes (/).
- The first character cannot be a number.

Input Format

- The first line contains a single integer `N` — the length of the string.
- The second line contains the string `str` of length `N`.

Output Format

- Print a single integer — 1 if the password is valid, 0 otherwise.

Constraints

- $1 \leq N \leq 1000$
- `str` may contain letters, digits, spaces, and the `/` character.

Sample Input / Output

Example 1

Input:

10

aB1_strong

Output:

1

Example 2

Input:

4

aB 1

Output:

0

Explanation

In Example 1, "aB1_strong" has length 10 (≥ 4), starts with a letter (a), contains a digit (1), contains a capital letter (B), and has no spaces or slashes — so all conditions are satisfied, and the output is 1.

In Example 2, "aB 1" contains a space character, which immediately violates the "no spaces or slashes" rule, so the output is 0, regardless of the other conditions being met.

Approach to Solve This Problem:

Translate each rule into a direct check, and fail fast wherever possible so we don't do unnecessary work.

1. **Check the length first:** If $n < 4$, the password is immediately invalid — return 0 without inspecting any characters.
2. **Check the first character:** If `str.charAt(0)` is a digit, return 0 immediately, since the first character cannot be a number.
3. **Scan the string once:** Walk through every character and:
 - If it is a space or a /, the password is invalid — return 0 right away.
 - If it is a digit, mark `hasDigit = true`.
 - If it is an uppercase letter, mark `hasCapital = true`.
4. **Final check:** After the scan, if both `hasDigit` and `hasCapital` are true, the password is valid — return 1. Otherwise, return 0.

Java Solution:

```
import java.util.Scanner;

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = Integer.parseInt(sc.nextLine().trim());
        String s = sc.nextLine();

        System.out.println(checkPassword(s, n));
    }

    static int checkPassword(String s, int n) {
        // Condition 1: Length check
        if (n < 4) {
            return 0;
        }

        // Condition 5: First character cannot be a digit
        if (Character.isDigit(s.charAt(0))) {
            return 0;
        }

        boolean hasDigit = false;
        boolean hasCapital = false;

        for (int i = 0; i < n; i++) {
            char ch = s.charAt(i);

            // Condition 4: No spaces or slashes allowed
            if (ch == ' ' || ch == '/') {
                return 0;
            }

            // Condition 2: Digit check
            if (Character.isDigit(ch)) {
                hasDigit = true;
            }

            // Condition 3: Capital letter check
            if (Character.isUpperCase(ch)) {
                hasCapital = true;
            }
        }

        if (hasDigit && hasCapital) {
            return 1;
        } else {
            return 0;
        }
    }
}
```

Solution:

1. **Read the input:** main() reads N, then reads the string str, matching the Input Format.
2. **Length check:** If $n < 4$, return 0 immediately — no need to inspect the string further.
3. **First character check:** If the first character is a digit, return 0 immediately, since this rule cannot be satisfied later in the string.
4. **Single pass through the string:** For each character, check for forbidden characters (space or /) first, returning 0 on the spot if found. Otherwise, update hasDigit and hasCapital flags using Character.isDigit() and Character.isUpperCase().
5. **Final decision:** Once the loop completes without an early return, both flags are checked together — only if both are true is the password considered valid, printed as 1; otherwise 0 is printed.

Complexity Analysis:

Time Complexity: $O(N)$

The string is scanned at most once (with early exits on failure), so the total work is linear in the length of the password.

Space Complexity: $O(N)$

Only a constant number of variables (hasDigit, hasCapital, loop index) are used; no auxiliary data structures are needed.